Computer Network I

Reti di Calcolatori I

Università di Napoli Federico II – Scuola Politecnica e delle Scienze di Base
Corso di Laurea in Informatica

Riccardo Caccavale

(riccardo.caccavale@unina.it)



UNIVERSITÀ DEGLI STUDI DI NAPOLI
FEDERICO II



DIETI

UNINA

Network Layer

Routing: Introduction

- We recall that, on packet receival, a **router can perform 3 actions**:
  - **Forwarding the packet** as is to a suitable neighbor.
  - **Forwarding a modified/replaced packet** (e.g., fragmentation, masquerading, etc.).
  - **Dropping the packet** (e.g., if expired).

- There are **different types of routers**, some are very simple and are used by local administrators (for instance to connect LANs and WANs), others are used by ISPs to allow packets to be forwarded to the right device in the entire Internet.

- As we saw**, routers are endowed with forwarding tables** that specify the local forward strategy (in the neighborhood of the router). The way these tables are created can be centralized or decentralized.
  - Routers have no knowledge about the rest of the network (at least on startup).

Network Layer

Routing: Introduction

- To decide where packets should be forwarded routers make use of **routing algorithms**, which determine good paths (i.e., sequence of routes) from senders to receivers.

- Typically, **a "good" path is one that has the minimal cost** (shortest or fastest path, etc.), in real-world **there are also policy issues** to be considered (e.g., rules specifying if an organization can talk with another etc.).

- The **routing problem is crucial for computer networks**, but the general problem of finding "good paths" on a network (or a graph) is particularly relevant not only for computer networks (e.g., videogames, autonomous driving, robotics, etc.).

Network Layer

Routing: Flooding

- Most simple approach is **flooding**: each router forwards all incoming packets to all links except for the one by which the packet is received.
  - All paths are explored (optimal one included), so the **packet will surely be delivered** if a path exists.
  - Packets going nowhere will eventually reach their **hop limit** and be discarded.
  - It is **tremendously robust** due to the redundancy of generated packets (e.g., if some routers are offline alternative paths are always considered).

- Flooding can be used in specific situations where redundancy is needed (e.g., broadcast of messages, military scenarios, etc.) but **it is impractical in high-performance and large networks** (such as Internet).

- For typical computer networks a more sophisticated routing algorithm is needed.

Network Layer

Routing: Problem Formulation

- We can **formalize** our network as a graph $G = (N, E)$, where:
  - N is a set of **nodes** (the routers of our network).
  - $E \subseteq N \times N$ is a set of **edges** (physical links) represented as a pair of nodes.

- An **edge also has a value representing its cost**, which may reflect the physical length of the link, the link speed, or the monetary cost associated with a link.

- We can formalize a generic **cost** as a function $c: N \times N \rightarrow \mathbb{R}$ such that, given a generic couple of nodes $(x, y) \in N \times N$:
  - $\forall x \in N$, then $c(x, x) = 0$
  - If $(x, y) \notin E$, then $c(x, y) = \infty$
  - If $(x, y) \in E$, then also $(y, x) \in E$ (undirected graph) and $c(x, y) = c(y, x)$

Network Layer

Routing: Problem Formulation

- Following this formulation, a **path** π in G is a **sequence of nodes** $\pi = (x_1, x_2, \dots, x_n)$ such that **all adjacent nodes are linked**, formally:

$$\forall x_i, x_{i+1} \in \pi, (x_i, x_{i+1}) \in E$$

- The **overall cost of the path** is the sum of the links' costs:

$$c(\pi) = \sum_{i=1}^{n-1} c(x_i, x_{i+1})$$

- Let's call $P(x, y)$ the **set of all possible paths** connecting the nodes x and y, we can define **the best path** as the path $\pi^* \in P(x, y)$ having minimum cost:

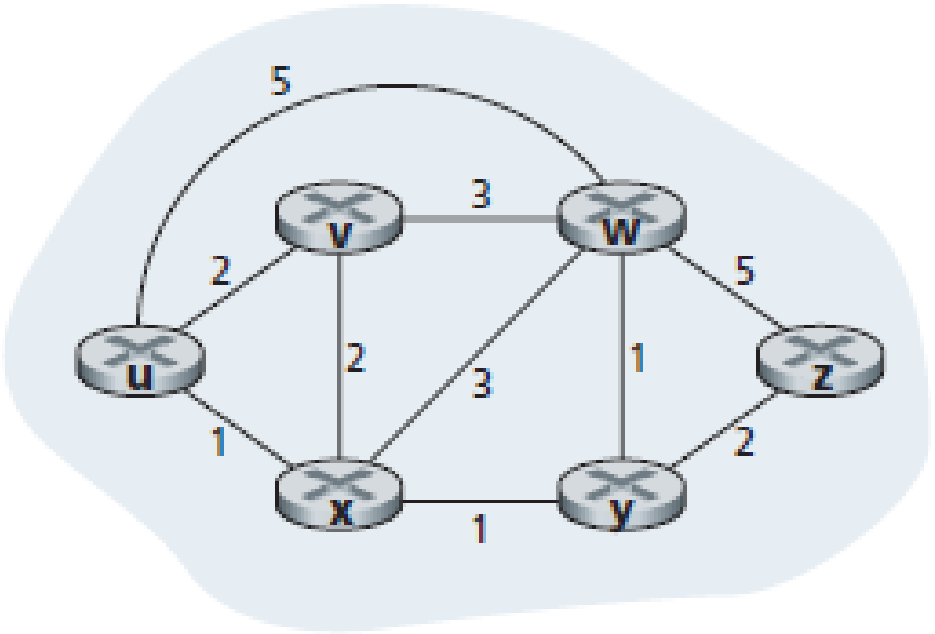
$$\forall \pi \in P(x, y), c(\pi^*) \leq c(\pi)$$

Network Layer

Routing: Problem Formulation

- In this example we have a graph composed of 5 nodes:
  - $N = \{u, v, w, x, y, z\}$
  - $E = \{(u, w), (u, v), (w, x), \ldots\}$
  - $c(u, w) = 5,\ c(u, v) = 2,\ c(u, x) = 1, \ldots$

- For instance, the best path π^* between nodes x and z is:
  - $\pi^* = (x, y, z)$
  - $c(x, y) = 1,\ c(y, z) = 2$
  - $c(\pi^*) = 3$



Network Layer

Routing: Problem Formulation

- Algorithms able to find such optimal path π^* between two nodes are called **shortest path algorithms**.

- In the case of computer networks, we are interested in **finding such optimal path** not for just two nodes but **for all possible couples of nodes**.

- A routing algorithm **converges** when the shortest path for all couples of nodes in the network is found.

- On networks **there are 2 families of algorithms** that are typically used:
  - Distance-Vector.
  - Link-State.

Network Layer

Routing: Distance-Vector Algorithm

- The **Distance-Vector** (DV) algorithm is a **decentralized** approach that exploit **local knowledge** about the network combined with nodes communication to find the shortest paths.
  - It is based on the **Bellman-Ford algorithm** that is used to compute paths.

- Here **each node receives some information** from one or more of its directly attached neighbors, performs a calculation, and then **distributes the results** of its calculation back to its neighbors.

- This algorithm is **asynchronous** in that it does not require all of the nodes to be coordinated with each other.

Network Layer

Routing: Distance-Vector Algorithm

- Let's call $d^*(x, y)$ be the cost (distance) of the best path from node x to node y. From the **Bellman equation** we can derive the following one:

$$d^*(x, y) = \min_{(x,\alpha)\in E}\{c(x, \alpha) + d^*(\alpha, y)\}$$

- This is quite intuitive, **if x is part of the best path, there must be and adjacent node α that also lies on the best path**, and the cost of the best path is given the cost of reaching α plus the best path between α and y.

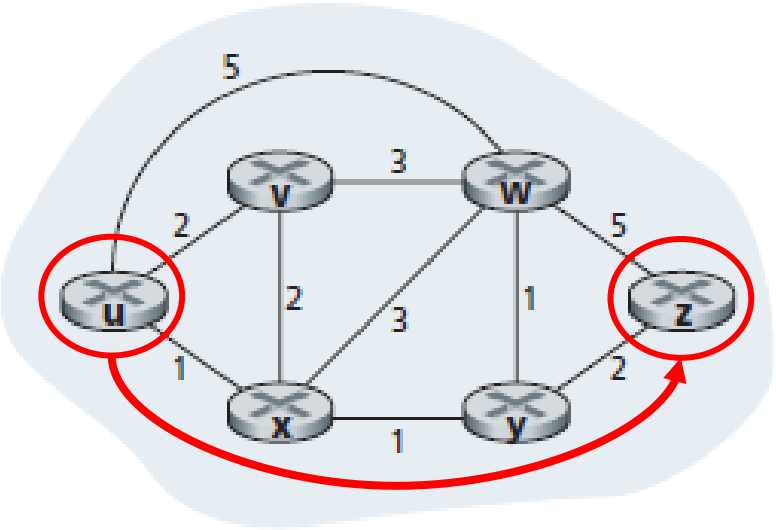
- If we are able to find such **α we can just add it into the forwarding table** and forward to it all packets directed to y.

- To do that, **we should have an estimation of $d^*(\alpha, y)$** for all neighbors (the distance vector).

Network Layer

Routing: Distance-Vector Algorithm

- Let's try with a real example. Considering u and z, the best path is:
  - $\pi^* = (u, x, y, z)$,
  - $d^*(u, z) = c(\pi^*) = 4$

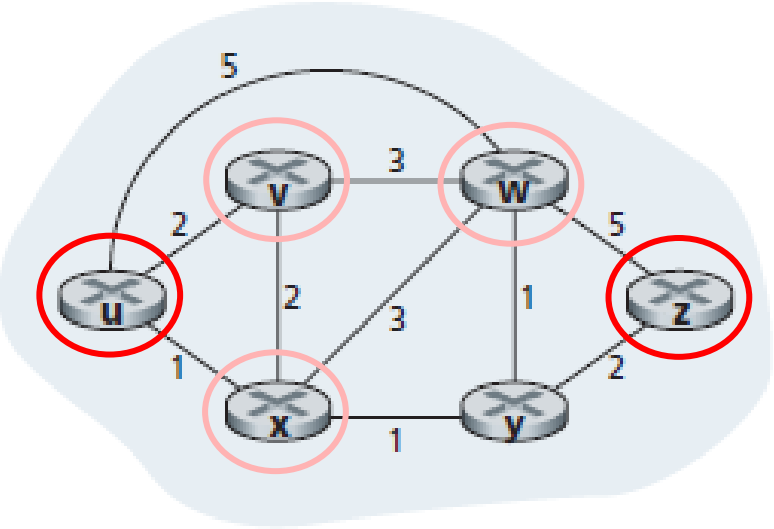


Network Layer

Routing: Distance-Vector Algorithm

- Let's try with a real example. Considering u and z, the best path is:
  - $\pi^* = (u, x, y, z)$,
  - $d^*(u, z) = c(\pi^*) = 4$

- Now, **for all the nodes along this optimal path the Bellman equation must stand**.
  - Let's see what happens for node u:
    - $d^*(u, z) = \min_{(u,\alpha)\in E}\{c(u, \alpha) + d^*(\alpha, z)\}$
  - In this example u has 3 neighbors (w, v, and x):
    - For $\alpha = w$ then $c(u, w) = 5$ and $d^*(u, z) = 3$.
    - For $\alpha = v$ then $c(u, v) = 2$ and $d^*(v, z) = 5$.
    - For $\alpha = x$ then $c(u, x) = 1$ and $d^*(u, z) = 3$
  - Node x **minimizes the equation, and it is the next one on the optimal path π^*!**



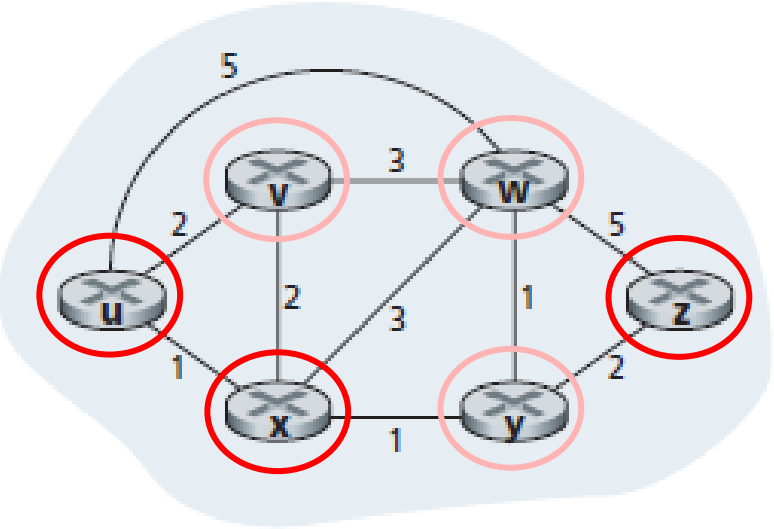
Network Layer

Routing: Distance-Vector Algorithm

- Let's try with a real example. Considering u and z, the best path is:
  - $\pi^* = (u, x, y, z)$,
  - $d^*(u, z) = c(\pi^*) = 4$

- We can **iterate this process on the next node** of the path (which is x) and we will see that **the Bellman equation stands** (i.e., that node y, which is next in π^*, will also minimize the equation, and so on).

- Therefore, if we have the distance vector d^* for all nodes, **we can easily use Bellman equation to compute next hop** in the path (forwarding table).



Network Layer

Routing: Distance-Vector Algorithm

- The pseudocode:

```
DistanceVector(x)
 for all nodes v
  if v is a neighbor of x then
    D_x(v) = c(x,v)
  else
    D_x(v) = ∞
 for all neighbors w and destinations y
  D_w(y) = ?
 send initial D_x(·) to all neighbors

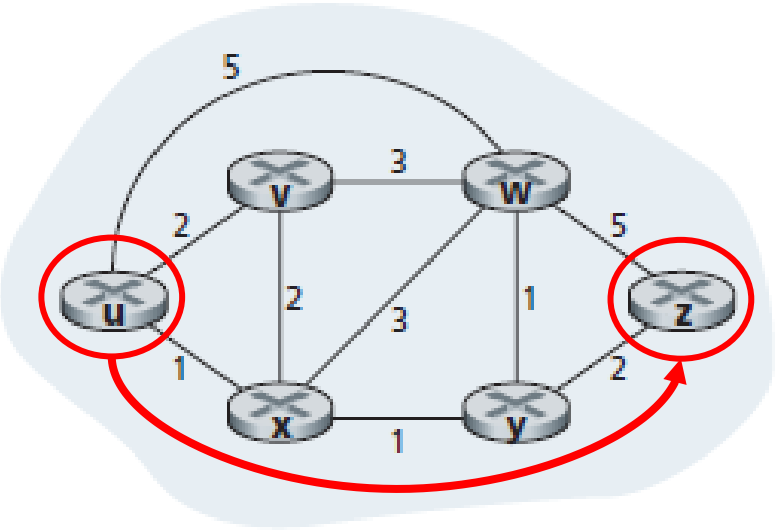
 repeat
  if cost of a neighbor updated then
   for all nodes y
    D_x(y) = min_v{c(x,v) + D_v(y)}
   if D_x(y) changed for any destination y then
    send updated D_x(·) to all neighbors
 until false //forever
```

- We use the **data-structure $D_w(v)$** to store the distance vectors from all nodes w to target nodes v.
- During the **initialization** phase:
  - Only **distances to neighbors are set**.
- During the **online** phase:
  - The **news** from the neighborhood are received.
  - Distance vectors are **updated** (Bellman equation).
  - Local changes are **communicated** to adjacent nodes.

Network Layer

Routing: Distance-Vector Algorithm

- To clarify this process **let's focus again on the path between u and z**:
  1. On start, **u only knows its neighbors**. The cost to z is infinity ($D_u(z) = \infty$).
  2. Node **y wakes up** and discovers a better path to z, which is the direct link y-z with cost 2 ($D_y(z) = c(y,z) = 2$). It communicates the good news to x.
  3. Node **x discovers that the best path to z now passes through y** with cost 3 ($D_x(z) = c(x,y) + D_y(z) = 1 + 2$), and forwards the good news to u.
  4. Node **u receives the news**. Now there is a path to z that costs less than infinity passing through x ($D_u(z) = c(u,x) + D_x(z) = 1 + 3$).



- The **same process** takes place for all nodes/paths.

Network Layer

Routing: Distance-Vector Algorithm

- As for the computational complexity, every time the cost of a node is updated, all nodes have to re-evaluate their edges. This process leads to a typical **worst-case complexity of** $\boldsymbol{O}(|\boldsymbol{N}||\boldsymbol{E}|)$.

- Pros:
  - The **algorithm is asynchronous**, this facilitates the computation as routers can on-line and in any stage adapt to updated costs.
- Cons:
  - **Slow convergence**: updates spread slowly as all nodes have to detect the change and send an updated to the adjacent ones.
  - **Count-to-infinity**: while improved paths are immediately recognized and adopted by the network, failures are detected slowly and may produce loops.